CSCI1409 Deep Learning

Week 3

Multi-Layer Perceptrons + Loss Functions

Aidyn Abirov

KBTU, School of IT and Engineering

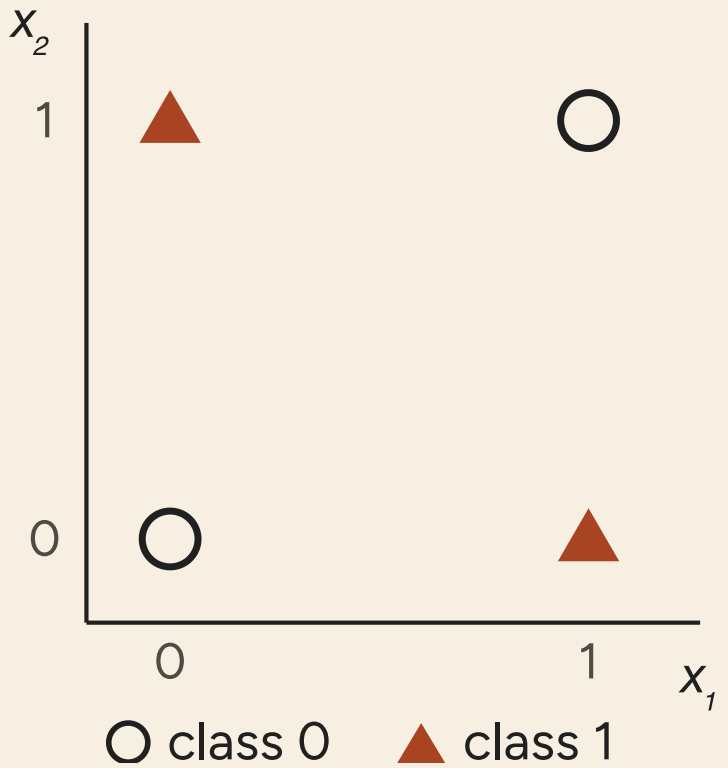
What Week 2 gives us

A single perceptron has a linear decision boundary.

A hidden ReLU unit composes familiar operations.

Backward computes gradients; an update changes parameters.

Why did the single perceptron fail on XOR?



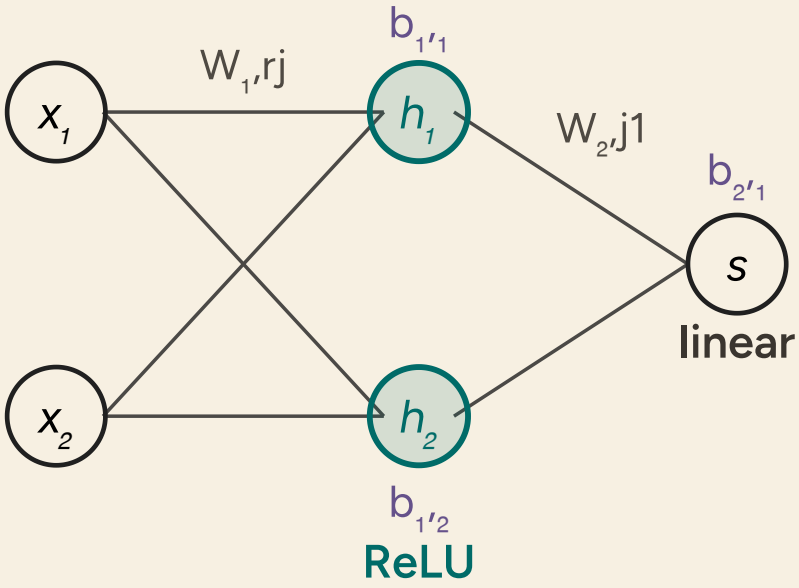
A multilayer perceptron

Two input features → two hidden units → one output

$$h_j = \text{ReLU}\left(\sum_{r=1}^{2} x_r W_{1,rj} + b_{1,j} \right)$$

$$s = \sum_{j=1}^{2} h_j W_{2,j1} + b_{2,1}$$

One hidden layer. Two trainable affine layers.

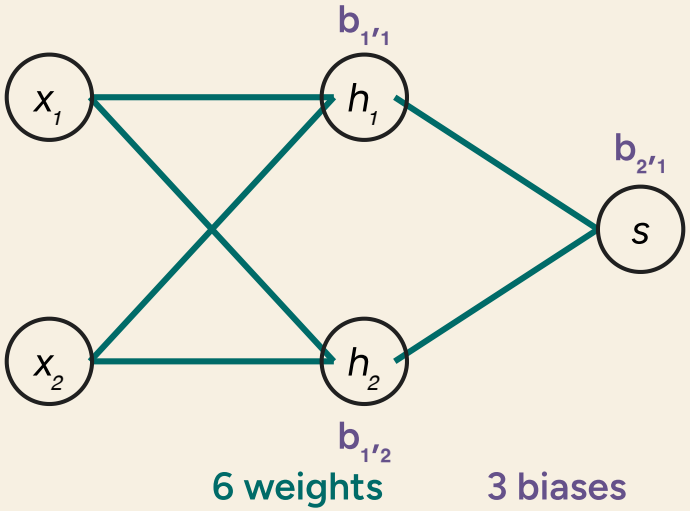


Count the trainable parameters

Part	Weights	Biases
Input → hidden	2 × 2 = 4	2
Hidden → output	2 × 1 = 2	1

4 + 2 + 2 + 1 = 9

Does using 100 examples instead of 4 change this count?



Two affine layers are still affine

Remove the activation:

$h = w_1x + b_1$ → $\hat{y} = w_2h + b_2$ = $\hat{y} = Ax + B$

$$\hat{y} = (w_2w_1)x + (w_2b_1 + b_2)$$

$A = w_2w_1$ $B = w_2b_1 + b_2$

More affine layers alone do not create a nonlinear function.

The same argument for matrices

$$H = XW_1 + b_1 \qquad \hat{Y} = HW_2 + b_2$$

$$\hat{Y} = X \underbrace{(W_1 W_2)}_{\text{effective weights}} + \underbrace{(b_1 W_2 + b_2)}_{\text{effective bias}}$$

The effective weights and bias still define one affine map.

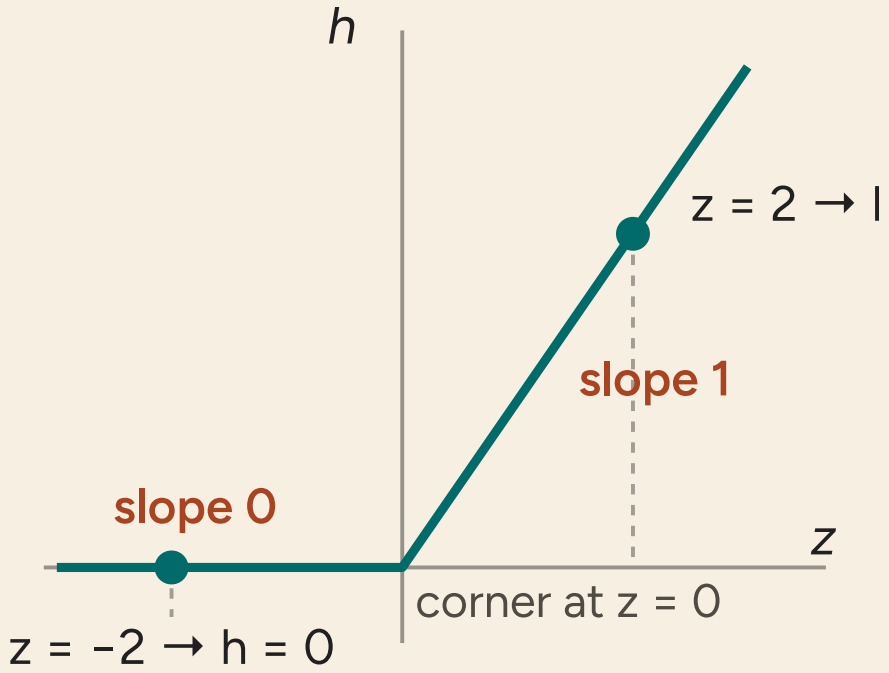
ReLU creates different regions

$$\mathrm{ReLU}(z) = \max(0, z)$$

Negative pre-activation: output 0.

Positive pre-activation: output z.

Different inputs can activate different units.

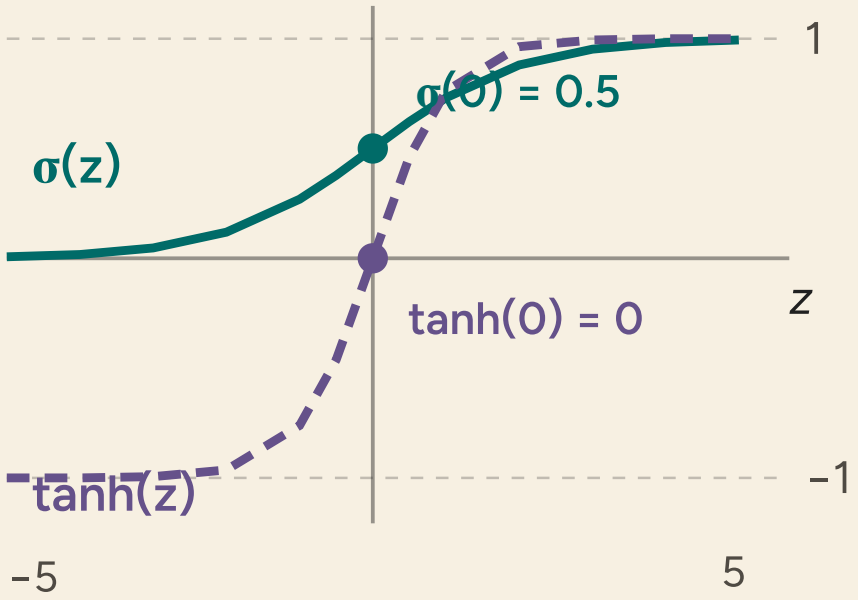


Hidden and output activations

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Activation	Output range
ReLU	$[0, \infty)$
Sigmoid	$(0, 1)$
Tanh	$(-1, 1)$

Hidden activation and output choice serve different roles.



Build two features for XOR

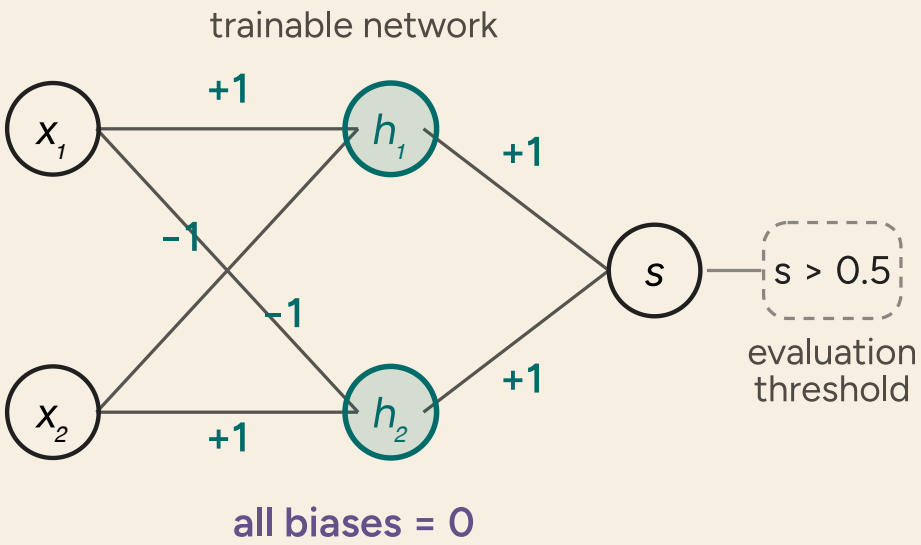
$h_1 = \text{ReLU}(x_1 - x_2)$

$h_2 = \text{ReLU}(x_2 - x_1)$

$s = h_1 + h_2$

Predict 1 if s > 0.5; otherwise predict 0.

Hand-selected weights demonstrate what the network can represent.



Compute the XOR forward pass

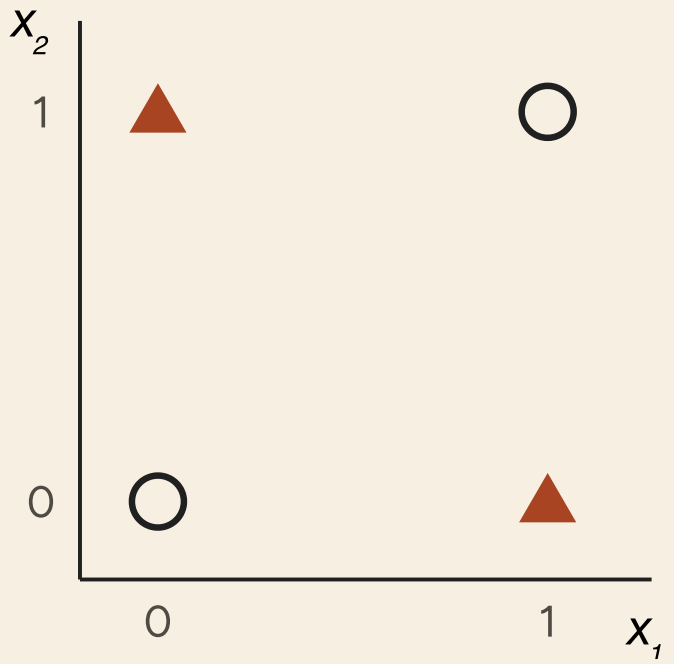
$h_1 = \mathrm{ReLU}(x_1 - x_2)$ $h_2 = \mathrm{ReLU}(x_2 - x_1)$ $s = h_1 + h_2$

Input (x_1, x_2)	h_1	h_2	Score s	Class
(0, 0)	0	0	0	0
(0, 1)	0	1	1	1
(1, 0)	1	0	1	1
(1, 1)	0	0	0	0

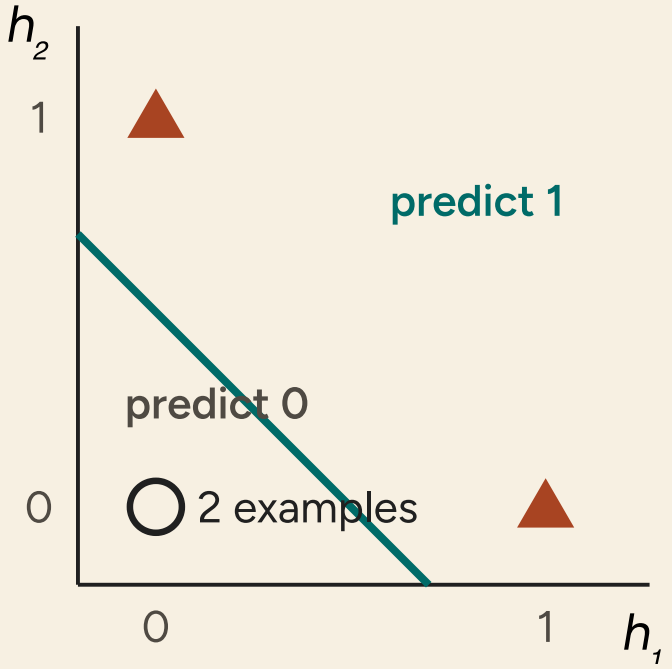
Which unit detects each positive example?

A new space makes separation possible

Input space: no single straight separator.

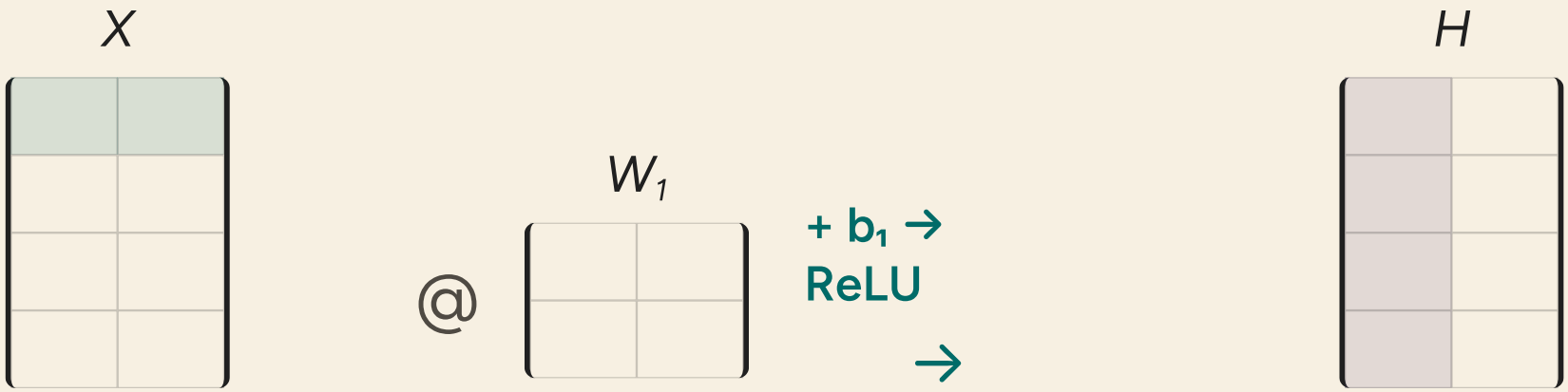


Hidden-feature space: $h_1 + h_2 = 0.5$



(0,0) and (1,1) → (0,0) (1,0) → (1,0) (0,1) → (0,1)

A whole batch through the network



one example = one row

(4, 2)

(2, 2)

one hidden unit = one column

(4, 2)

$$Z_1 = XW_1 + b_1 \quad H = \mathrm{ReLU}(Z_1) \quad Z_2 = HW_2 + b_2$$

This deck stores weights as input width × output width.

Check every shape

Quantity	General shape	N = 4, d = 2, m = 3, k = 1
X	(N, d)	(4, 2)
W_1 *and* b_1	(d, m) and (m,)	(2, 3) and (3,)
Z_1 *and* H	(N, m)	(4, 3)
W_2 *and* b_2	(m, k) and (k,)	(3, 1) and (1,)
Z_2	(N, k)	(4, 1)

Parameters = dm + m + mk + k = 13

Shared parameters and broadcast biases

$$Z_{1,ij} = \sum_{r=1}^{d} X_{ir} W_{1,rj} + b_{1,j}$$

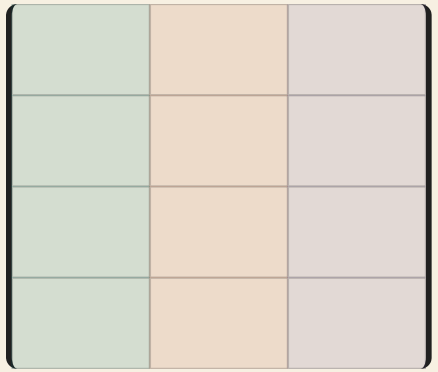
Each example uses the same W_1 and b_1.

One bias per hidden unit is added to every example row.

```
assert prediction.shape ==
target.shape
```

For elementwise regression errors: (N,1) must match (N,1).

Z_1 pre-activations (4 × 3)



j = 1 j = 2 j = 3

+

$b_{1,1}$	$b_{1,2}$	$b_{1,3}$

one bias per column, added to every row

Lecture 1 checkpoint

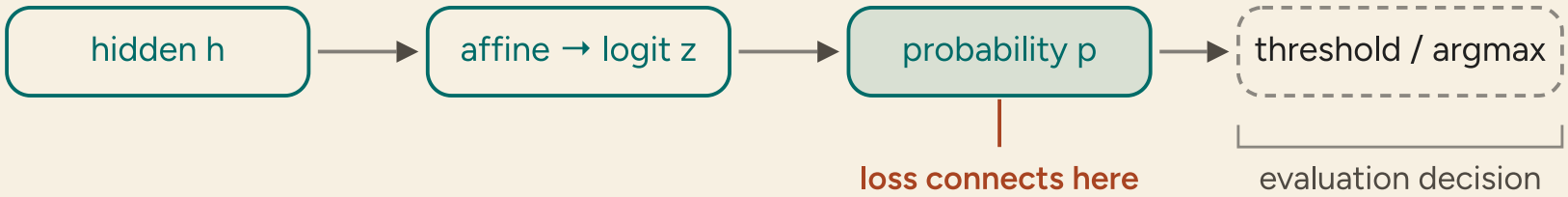
1. Why do two affine layers without an activation remain affine?

2. How many parameters does a 2–2–1 network have?

3. Is the XOR score s automatically a probability?

Score, probability or prediction

Quantity	Meaning
Hidden activation h	Intermediate feature
Logit z	Unrestricted classification score
Probability p	Output of sigmoid or softmax
Predicted class	Decision after threshold or argmax



Regression can use an unrestricted linear output directly.

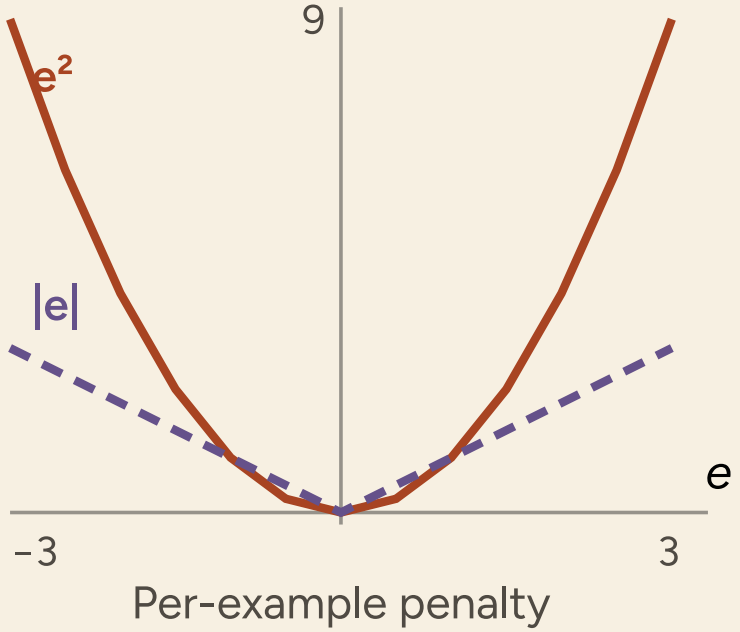
Two regression losses

$$e_i = \hat{y}_i - y_i$$

$$L_{\text{MSE}} = \frac{1}{N} \sum_i e_i^2$$

$$L_{\text{MAE}} = \frac{1}{N} \sum_i |e_i|$$

MSE squares each error. MAE takes its absolute value.



Per-example penalty

One large error changes the loss

Targets: [0, 1, 2, 3]

Predictions	Nonzero residual	MSE	MAE
[0, 1, 2, **7**]	4	4	1
[0, 1, 2, **11**]	8	16	2

Doubling this error quadruples its squared contribution.

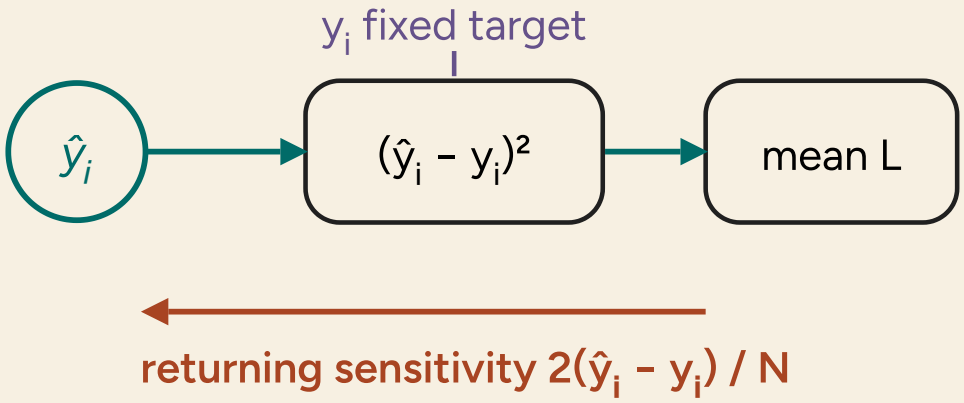
The loss starts the backward pass

For mean squared error:

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{2}{N}(\hat{y}_i - y_i)$$

If the prediction is too large, what is the gradient's sign?

Keep the batch-mean factor exactly once.



Binary cross entropy

$$p = \sigma(z) \qquad y \in \{0, 1\}$$

$$P(y \mid x) = p^{y}(1 - p)^{1-y}$$

y = 1 → label probability p y = 0 → label probability 1 − p

$$\ell = -\ln P(y \mid x) = -[\, y \ln p + (1 - y) \ln(1 - p) \,]$$

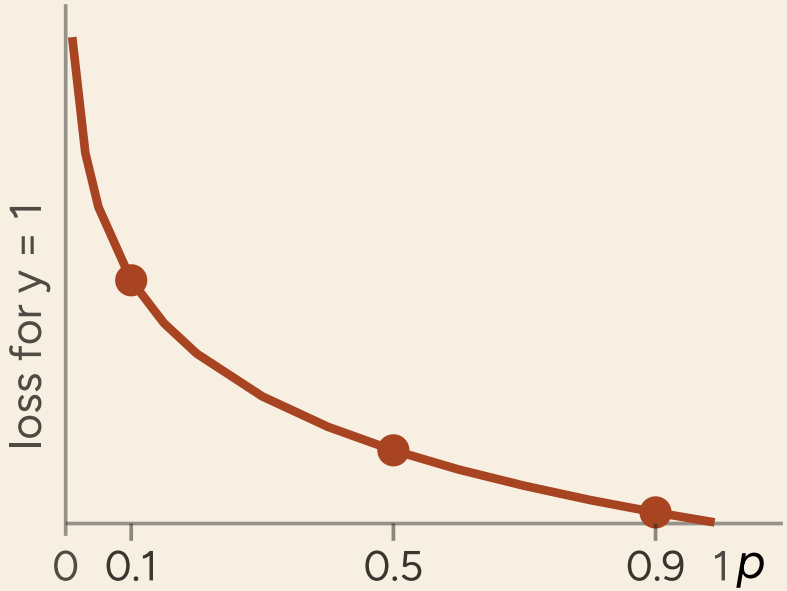
Minimize the negative log probability assigned to the observed label.

Confidence changes the penalty

For $y = 1$: $\ell = -\ln p$

Class-1 probability p	Loss
0.9	≈ 0.105361
0.5	≈ 0.693147
0.1	≈ 2.302585

Confidently wrong predictions receive a large penalty.



BCE and sigmoid give a simple gradient

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p} \qquad \frac{\partial p}{\partial z} = p(1 - p)$$

$$\frac{\partial \ell}{\partial z} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p}\right) p(1 - p) = \boldsymbol{p - y}$$

For a batch mean: $\frac{\partial L}{\partial z_i} = \frac{p_i - y_i}{N}$

The derivative is with respect to the logit; the sigmoid factor is already inside it.

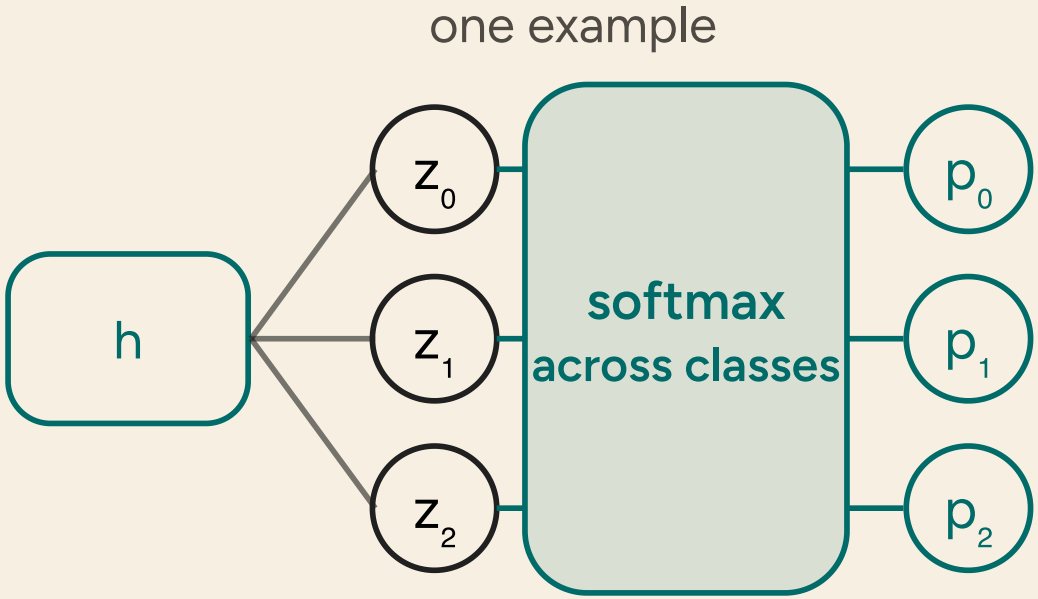
Several classes need several logits

Exactly one of K classes:

$$p_k = \frac{e^{z_k}}{\sum_{j=0} e^{z_j}}$$

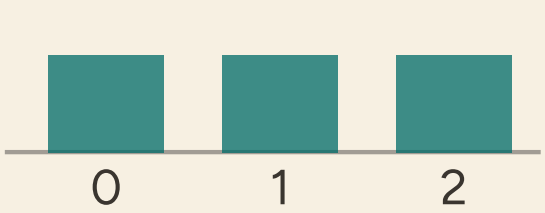
$$p_k > 0 \quad \sum_{k=0}^{K-1} p_k = 1$$

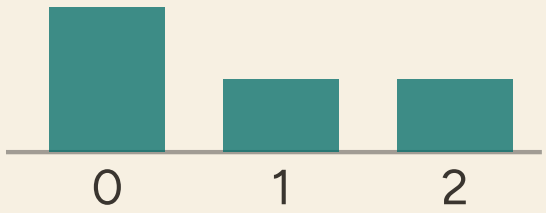
Normalize across classes for each example.



Compute softmax probabilities

Logits	Exponentials	Probabilities
[0, 0, 0]	[1, 1, 1]	[1/3, 1/3, 1/3]
[ln 2, 0, 0]	[2, 1, 1]	[0.5, 0.25, 0.25]





Each row is a separate example. Each probability row sums to 1.

Cross entropy uses the correct class

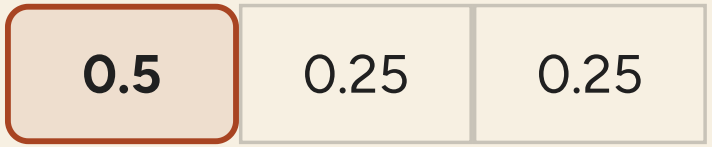
$$\ell = -\ln p_c$$

For p = [0.5, 0.25, 0.25]:

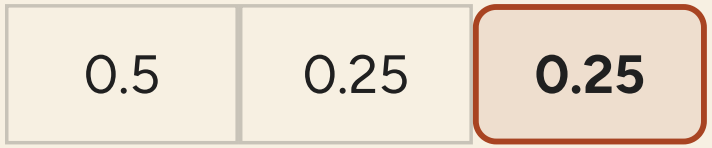
Correct class c	p_c	Loss
0	0.5	≈ 0.693147
2	0.25	≈ 1.386294

$$L = -\frac{1}{N}\sum_i \ln p_{i,c_i}$$

target c = 0



target c = 2



Match the task output and loss

Task	Output	Typical loss
Real-valued regression	Linear value	MSE or MAE
Binary classification	One logit → sigmoid	Binary cross-entropy
Exactly one of K classes	K logits → softmax	Multiclass cross-entropy

Choose the pair for temperature, spam detection, and cat/dog/bird.

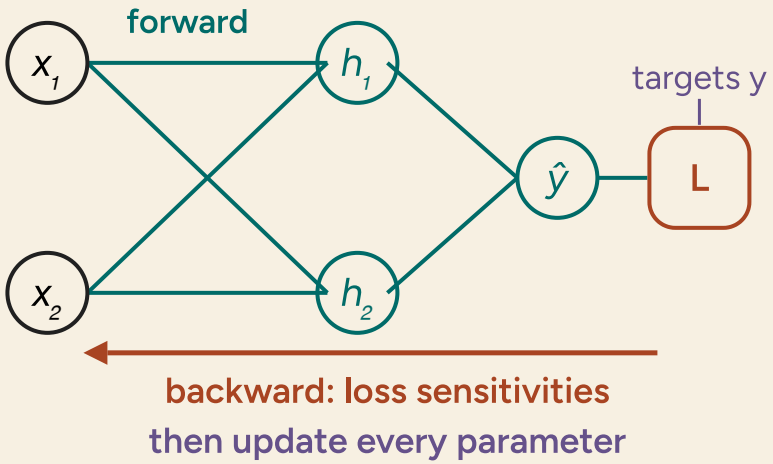
The same learning loop through an MLP

```
outputs = forward(inputs, parameters)
loss = objective(outputs, targets)
gradients = backward(loss)
parameters = parameters - learning_rate * gradients
```

The loss starts the signal. The chain rule carries it backward.

Compute all gradients before changing any parameter.

regression example



Find three training bugs

Proposed implementations — each contains a mistake

1. MSE residual: predictions `(N,1)` minus targets `(N,)`.
2. Sigmoid + mean BCE: logit gradient `(p-y)/N * p*(1-p)`.
3. BCE evaluated on thresholded 0/1 classes.

What would you change?

Three final checks

1. What lets an MLP represent a nonlinear function?
2. Can equal classification accuracy come with different BCE?
3. Does backward update the parameters?

Practice and Homework 3

Practice: affine layers, MSE/MAE, XOR, classification losses.

Homework 3: three tasks; AI only for Task 3.

Submit directly to me on Teams before the next lecture starts.

Individual homework defense during practice.

We have the gradients.

How should we choose the update?

APPENDIX · REFERENCE

Stable probability calculations

Stable softmax for one example

$$q_k = z_k - \max_j z_j \qquad p_k = \frac{e^{q_k}}{\sum_j e^{q_j}}$$

Binary cross-entropy directly from a logit

$$\ell = \max(z, 0) - yz + \ln(1 + e^{-|z|})$$

Compute with probabilities or with the equivalent combined logit formula.

APPENDIX · REFERENCE

Matrix backpropagation reference

One-output regression MLP with mean squared error

$$Z_1 = XW_1 + b_1 \qquad H = \mathrm{ReLU}(Z_1) \qquad \hat{Y} = HW_2 + b_2$$

$$G = \frac{2}{N}(\hat{Y} - y) \qquad dW_2 = H^{\mathsf{T}}G \qquad db_2 = \sum_i G_i$$

$$D = (GW_2^{\mathsf{T}}) \odot \mathbf{1}[Z_1 > 0] \qquad dW_1 = X^{\mathsf{T}}D \qquad db_1 = \sum_i D_i$$

Sum over example rows. Average once. Update afterward.

APPENDIX · REFERENCE

Losses as negative log likelihoods

Model of the target	Negative log likelihood
Gaussian with fixed variance	Squared error, up to scaling and constants
Bernoulli	Binary cross-entropy
Categorical	Multiclass cross-entropy

$$y \mid x \sim \mathcal{N}(f_\theta(x), \sigma^2) \quad \Rightarrow \quad -\ln P(y \mid x) = \frac{(y - f_\theta(x))^2}{2\sigma^2} + C$$

APPENDIX · REFERENCE

Numerical reference values

Lecture example	Reference
2–2–1 / 2–3–1 parameter counts	9 / 13
XOR scores in row order (0,0), (0,1), (1,0), (1,1)	[0, 1, 1, 0]
Residuals [0, 0, 0, 4]: MSE / MAE	4 / 1
Residuals [0, 0, 0, 8]: MSE / MAE	16 / 2
−ln(0.9) / −ln(0.5) / −ln(0.1)	≈ 0.105361 / 0.693147 / 2.302585
Uniform three-class cross-entropy	ln 3 ≈ 1.098612